

AI 코딩 활용 과정

구자환 초빙교수

jhkoo@skku.edu

<https://sites.google.com/site/jahwankoo>

Sungkyunkwan University

➤ 전체 개요

➤ 과정명

- AI 기반 코딩활용 특강 (30시간)

➤ 대상

- 국세공무원 (데이터 활용 및 AI 기초 이해가 필요한 실무자)

➤ 형태

- 실습 중심(Colab, Claude Desktop, Streamlit, VBA 병행)

➤ 주요 목표

➤ AI 코딩 환경 이해

- Colab 및 Claude Desktop 설치, ChatGPT vs Claude 비교, MCP 구조 이해

➤ Python 코딩 실습

- 조건문·반복문·리스트·함수 실습 및 Claude MCP 명령형 학습

➤ 데이터 처리 및 시각화

- Pandas, Matplotlib, Altair로 세수 데이터 분석 및 대시보드 구현

➤ 업무 자동화

- VBA 매크로 및 Claude MCP를 통한 Excel 자동화 실습

➤ 통합 미니프로젝트

- Colab + Claude + Streamlit + VBA를 연계한 자동화 보고서 생성 프로젝트

일정 계획

구분	주제	상세내용	시간	주제	상세내용	시간
AI 코딩 활용 (30H)	Colab 및 Claude Desktop 환경 소개	- Google Colab 소개 및 계정 연결 - Claude Desktop 설치 및 MCP 개념 이해 - ChatGPT / Claude 비교 및 병행 활용 전략 - AI 기반 코딩 학습(Vibe Coding) 개념	2	Colab + MCP를 활용한 웹앱 구현	- Streamlit 개요 - Claude MCP Filesystem 서버 실습 - ChatGPT로 코드 생성 후 Colab에서 시각화 - ngrok을 활용한 웹 실행	3
	Colab + Claude를 활용한 Python 기초	- 변수, 조건문, 반복문 실습 - 리스트, 함수 기초 및 자동 코드 생성 - ChatGPT 프롬프트 & Claude MCP 명령 비교 - 실시간 코드 피드백 경험	4	Claude MCP를 활용한 챗봇 구현	- ChatGPT API 및 Claude API 비교 - Claude MCP fetch 서버 실습 - Streamlit 기반 질의응답 챗봇 구현	4
	Colab을 활용한 데이터 전처리 및 MCP 연동	- Pandas 기본 문법 - CSV / JSON 파일 불러오기 및 전처리 - Claude MCP Filesystem으로 CSV 접근 - 결측치 처리 및 데이터 정제	3	Colab + Claude MCP 기반 데이터 대시보드	- 연도별 세수 데이터 분석 및 시각화 - Altair / Matplotlib 그래프 생성 - 실시간 대시보드 구축 및 Claude 응답 연결	3
	Colab + VBA 자동화 실습	- Excel VBA 매크로 개념 및 자동화 원리 - ChatGPT로 VBA 코드 생성 - Claude MCP로 로컬 데이터 접근 자동화 - 보고서 작성 매크로 실습	4	미니프로젝트 (Colab + Claude 통합)	- 세무 데이터 자동화 도구 제작 - Claude MCP를 통한 파일 입출력 연동	4
					- 예시 1: 세무 데이터 대시보드 - 예시 2: 엑셀 VBA 자동 보고서 - 예시 3: ChatGPT+MCP 통합 데모	3

➤ 기대 효과

➤ **AI 코딩 실습 역량 강화**

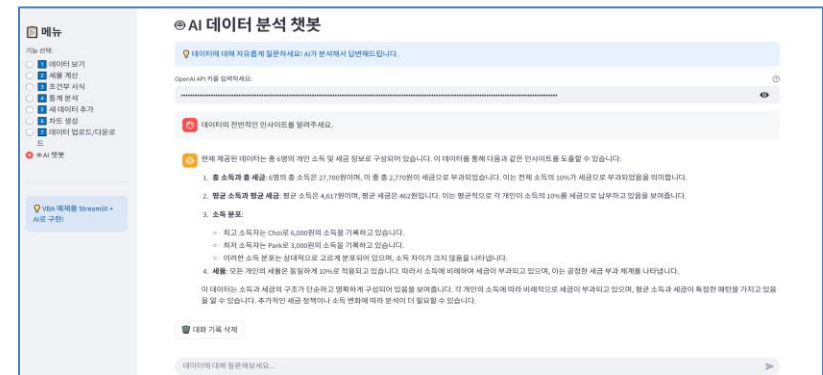
➤ **세무 데이터 분석 및 자동화 보고서 구현 능력 향상**

➤ **AI 기반 업무자동화 도입 가능성 탐색**

어디까지 할 수 있나?

➤ 데모

➤ <https://tax-analysis-app-chat-openai-emdsgqfwwjegku6vzychir.streamlit.app/>



Part 1.

Colab 및 Claude Desktop

환경 소개

(2H)

➤ 2024년 AI 키워드

LLM(Large Language Model)

➤ 2025년 AI 키워드

AI Agents

➤ AI 보조 코딩의 흐름

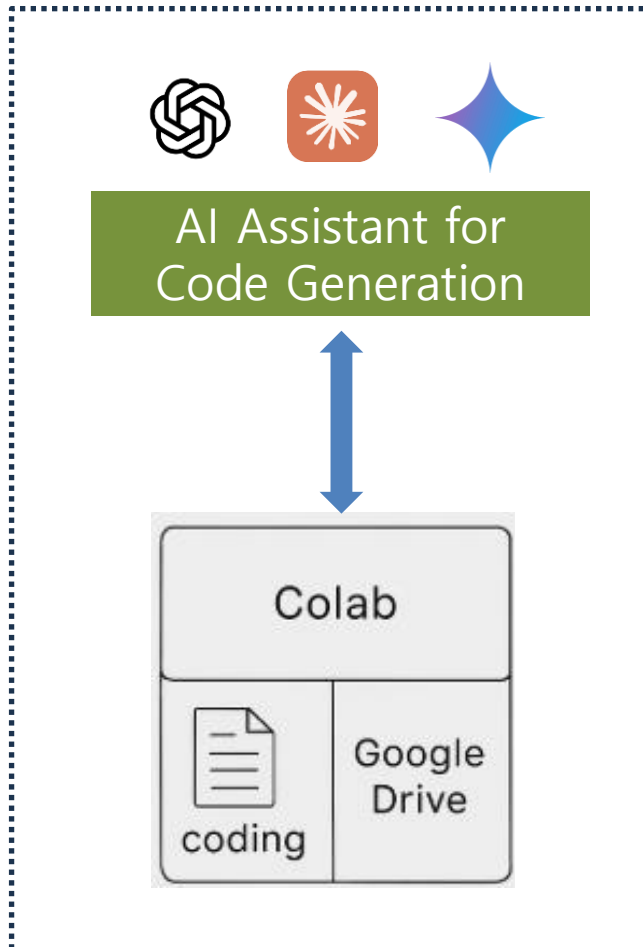
LLM + Agents

➤ "코드 보조"에서 "업무 지원" 으로 진화

Modern Software Development Style

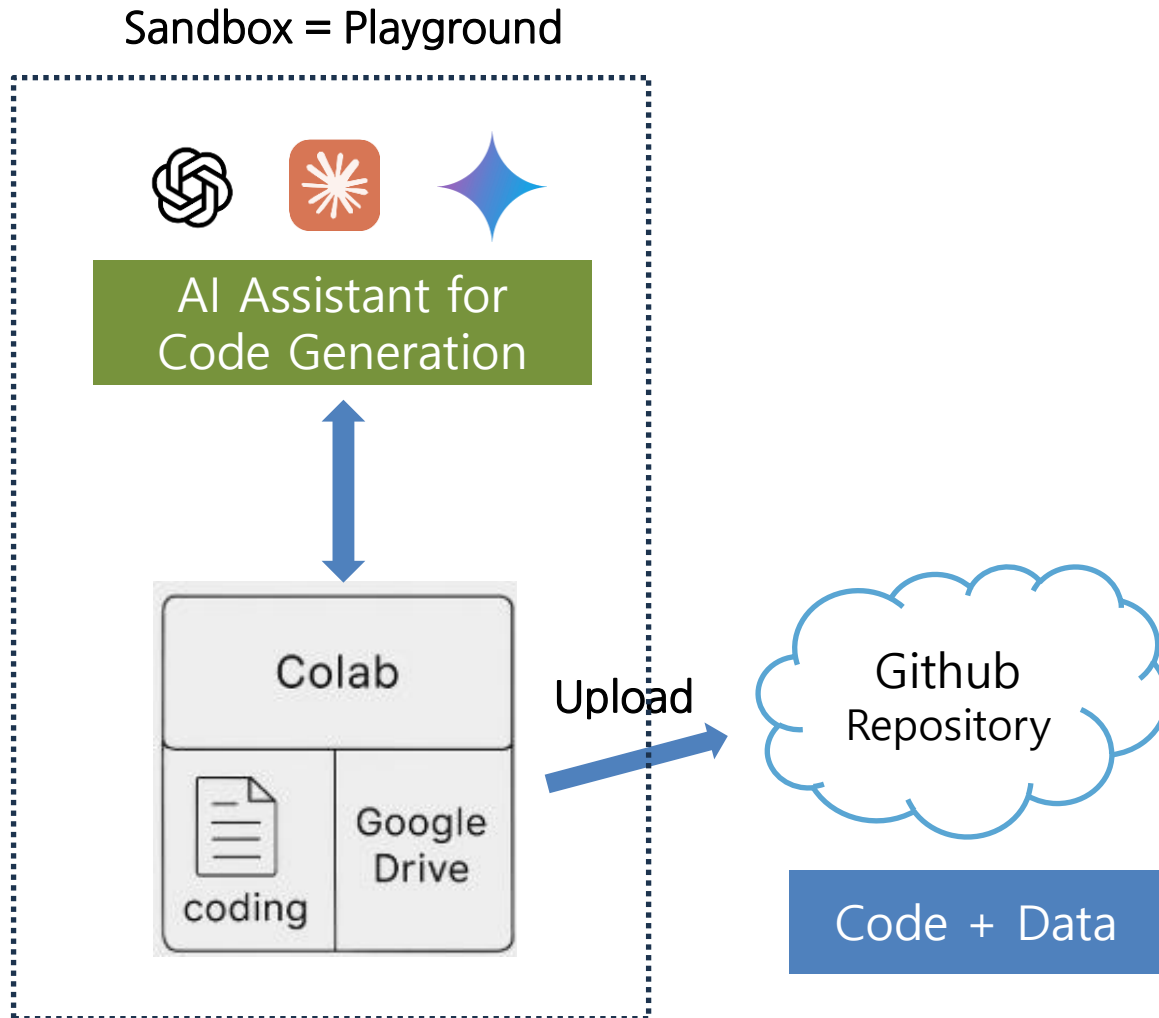
➤ Sandbox (= Playground)

Sandbox = Playground

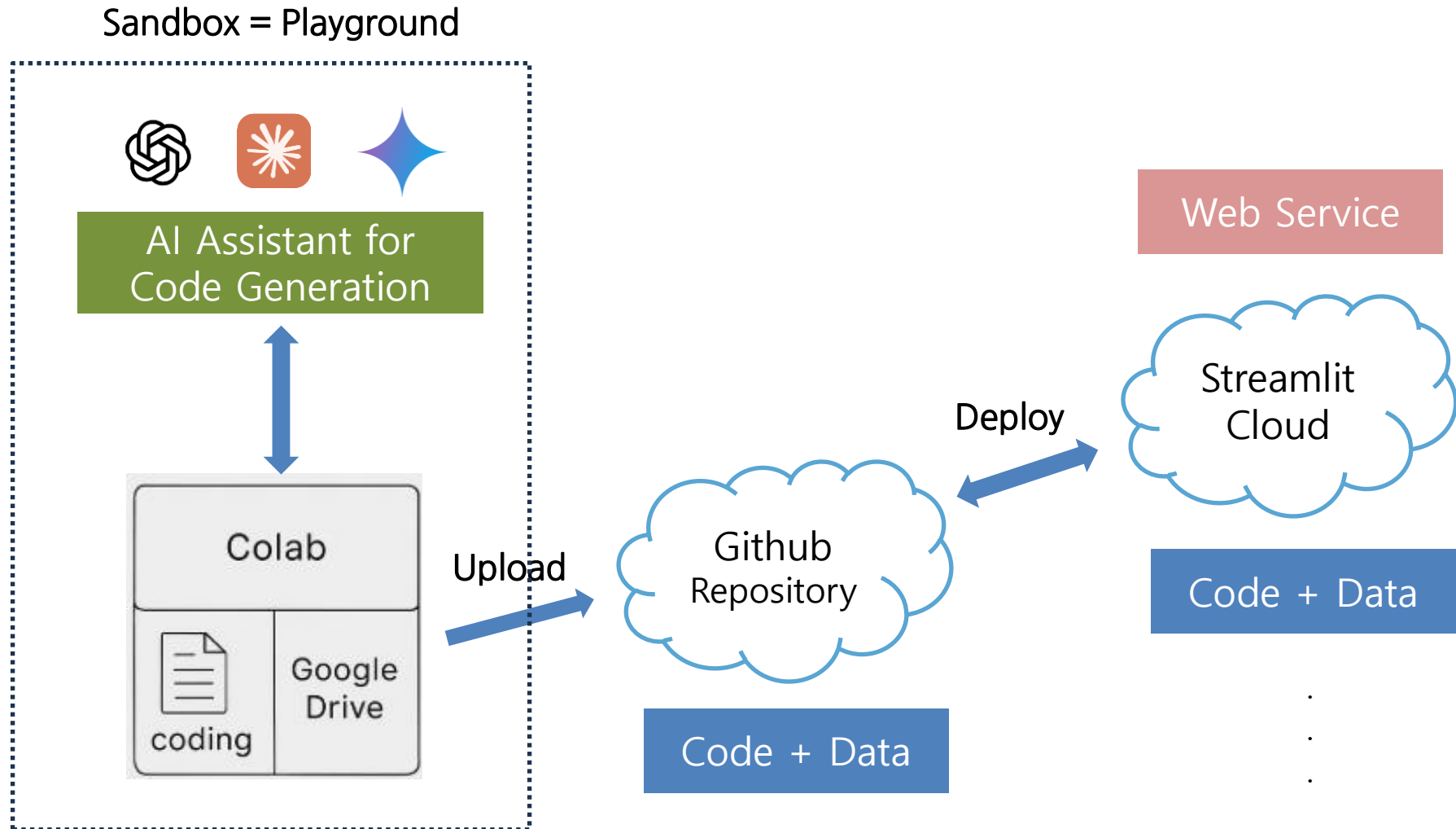


Modern Software Development Style

➤ Upload to GitHub

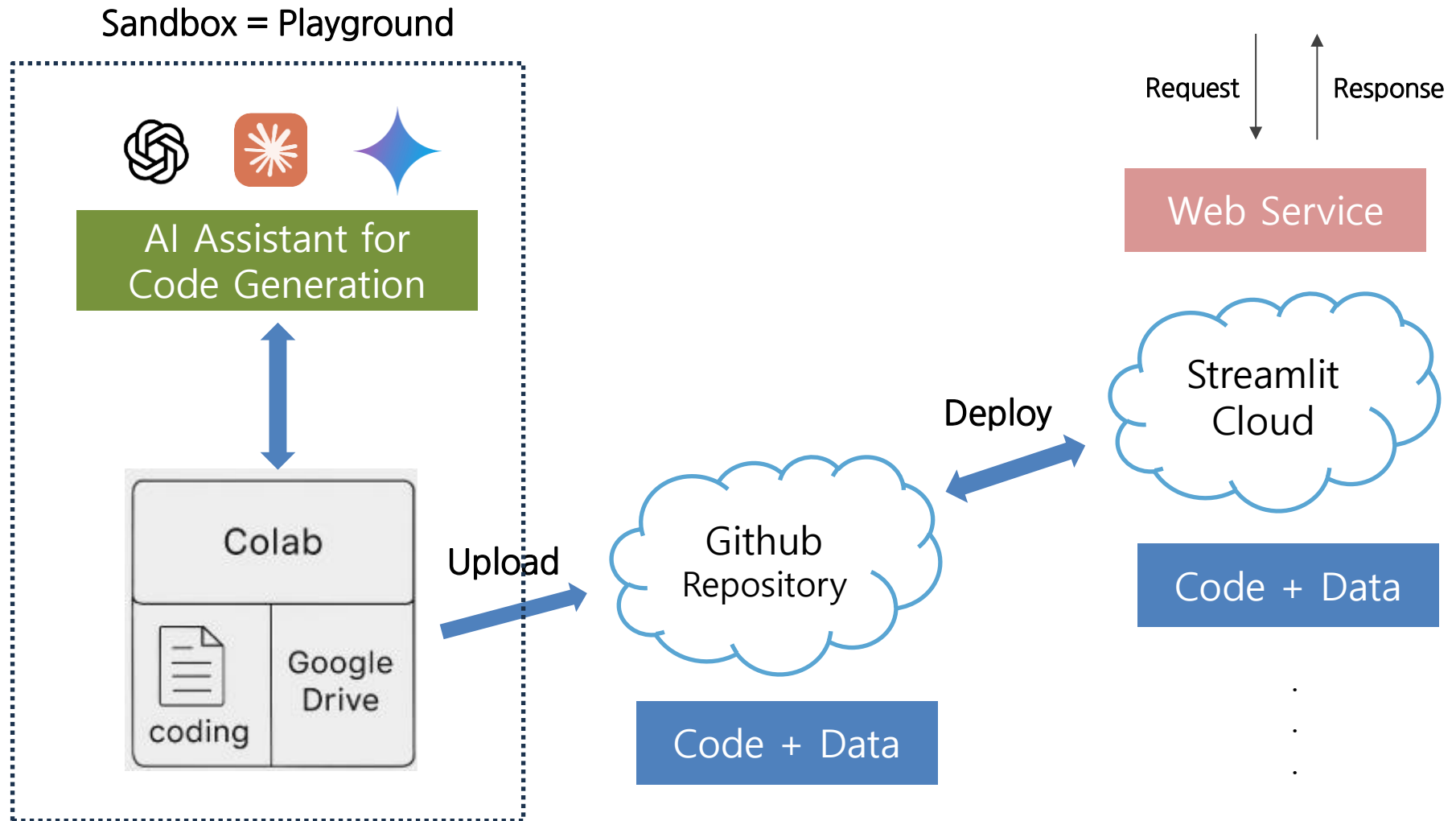


➤ Streamlit Cloud Deployment



Modern Software Development Style

➤ Web Service & Browser



MCP(Model Context Protocol)

➤ MCP란 무엇인가?

➤ Model Context Protocol

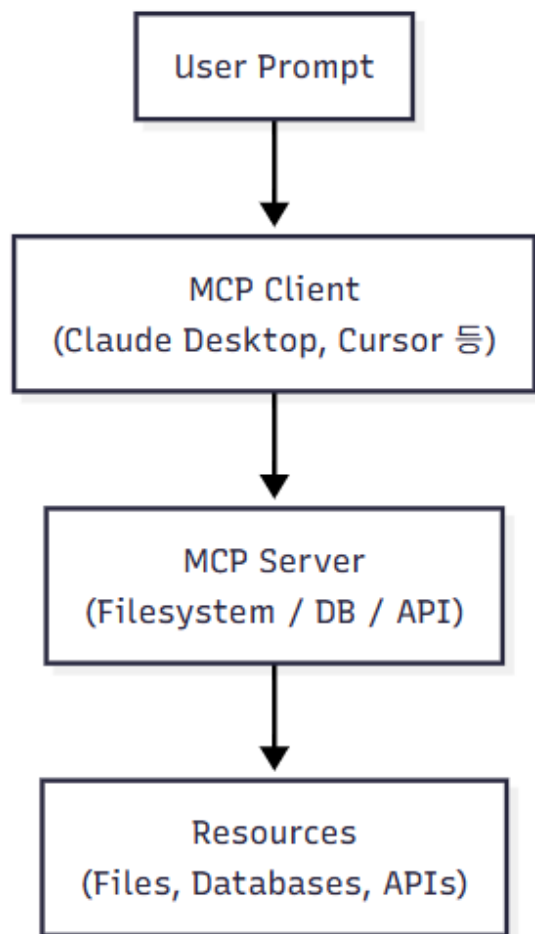
➤ AI 모델이 외부 자원에 **안전하고 표준화된 방식으로 접근**하도록 설계된 프로토콜

➤ Claude 같은 모델이 "내 PC의 파일 읽기", "DB에서 질의", "API로 데이터 가져오기" 등의 작업을 **직접 수행**할 수 있게 하는 연결 규칙

➤ MCP 아키텍처

구성요소	역할	예시
Client	사용자 요청을 AI 모델이 처리하도록 중개하는 역할	Claude Desktop, Cursor, Claude CLI
Server	실제 리소스(파일, DB, API)에 접근하여 데이터를 제공하는 엔드포인트	Filesystem Server, API Server, DB Server
Resources	AI가 접근하려는 실제 데이터 또는 외부 시스템	CSV, JSON, PostgreSQL, REST API

➤ MCP 아키텍처 다이어그램



1. 사용자 요청:

"세금 데이터 CSV 파일을 불러와 평균을 계산해줘."

2. Claude (Client)

→ MCP Filesystem Server에 요청 전송.

3. Server

→ 지정된 경로에서 `tax_data.csv` 파일 읽기.

4. Claude (Client)

→ 파일 내용을 받아 Pandas로 분석 후 결과 반환.

➤ MCP 서버 예시

- Filesystem: 로컬 파일 불러오기
- API: 공공데이터 API 호출
- DB: SQL 질의 실행

➤ MCP의 장점

- 데이터 활용성 극대화
- 행정업무 자동화 가능
- 실습: CSV 읽고 저장하기

➤ Claude Desktop 소개

- Anthropic 제공 대화형 AI 클라이언트
- MCP 지원 → 다양한 서버와 연동

➤ 설치 과정

➤ 설치 파일 다운로드 & 실행

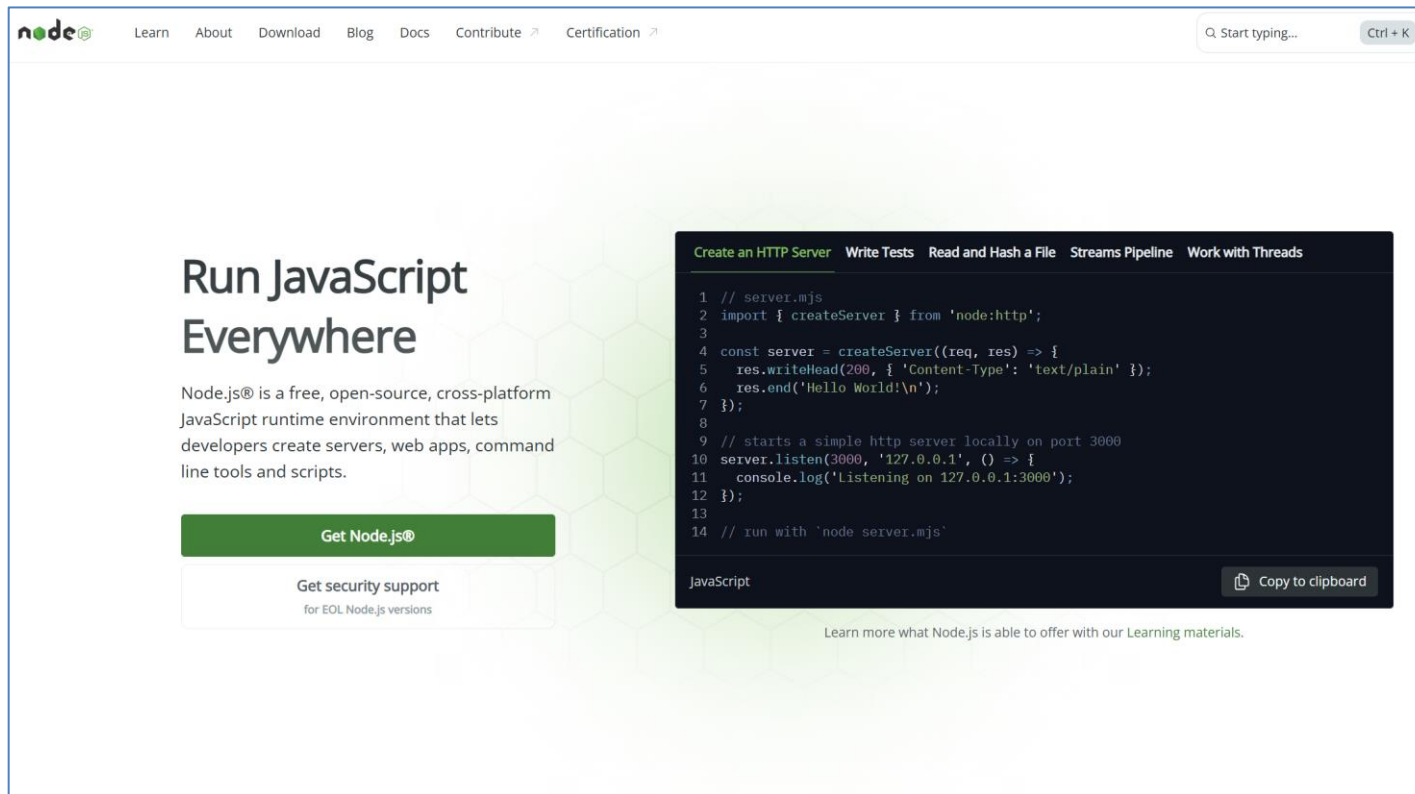
➤ <https://claude.ai/download>



➤ 설치 과정

➤ Node.js LTS 설치 및 확인

➤ <https://nodejs.org/en>



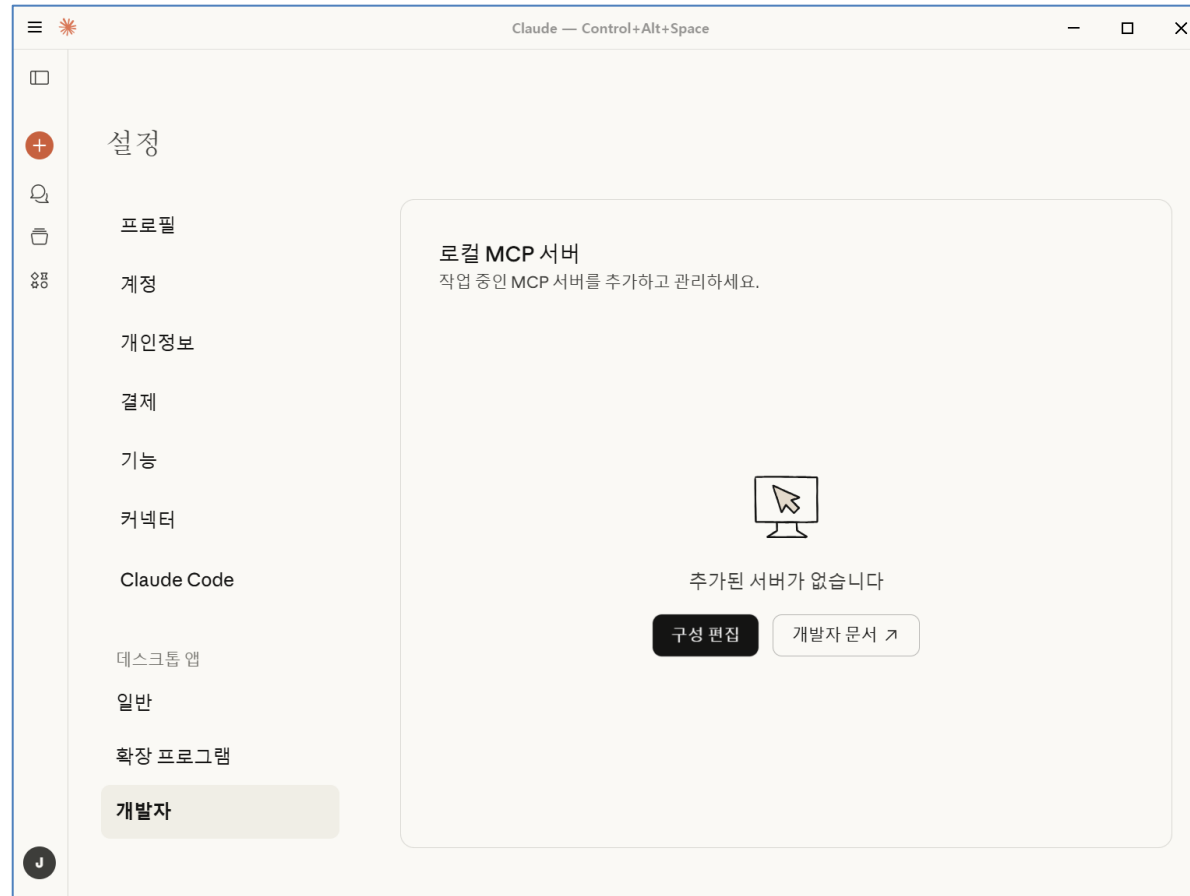
➤ 설치 과정

➤ .Wdata 폴더 만들기

```
C:\Users\jkhoo>mkdir .\mcp-demo\data  
C:\Users\jkhoo>cd mcp-demo  
C:\Users\jkhoo\mcp-demo>cd data  
C:\Users\jkhoo\mcp-demo\data>
```

➤ 설치 과정

➤ MCP 설정 파일 열기



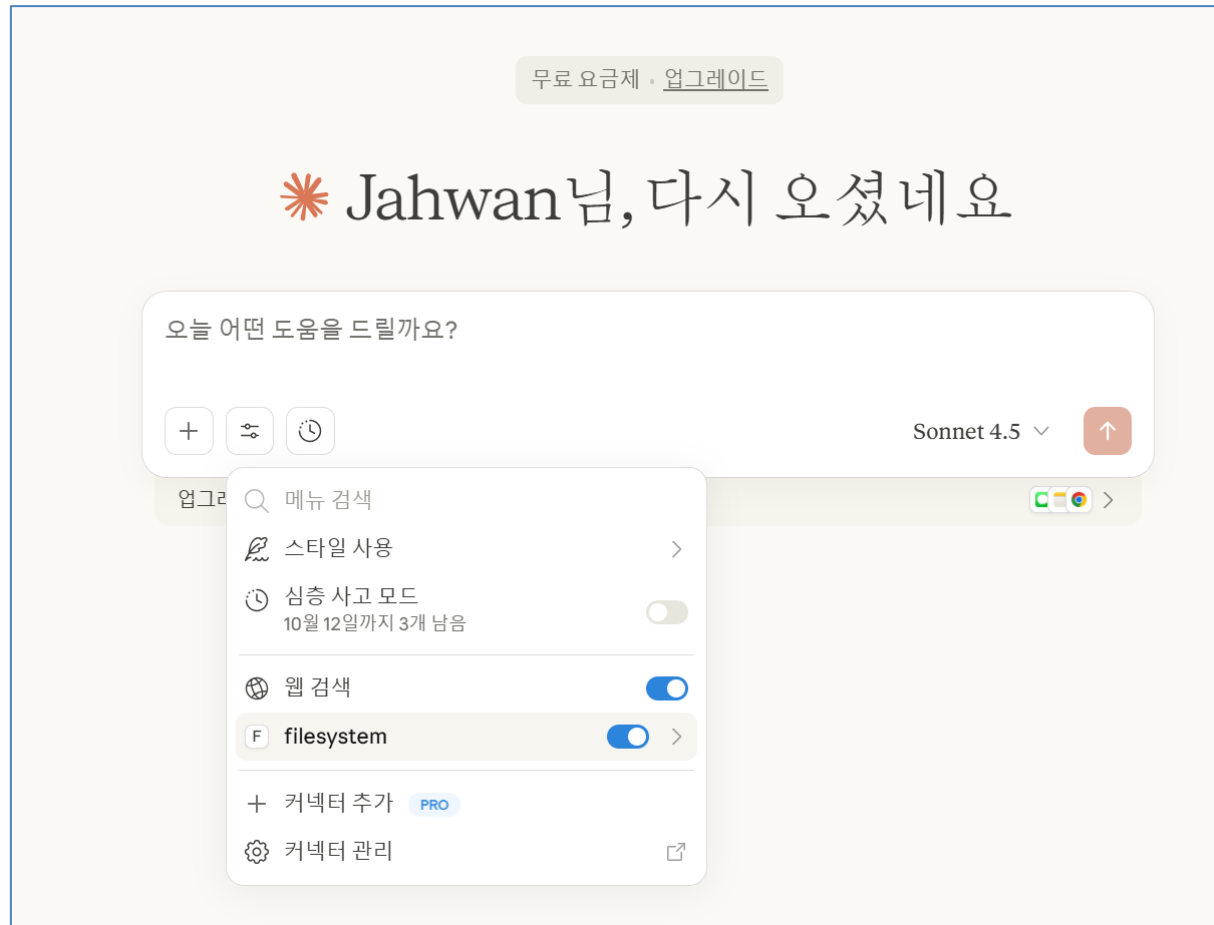
➤ 설치 과정

➤ claude_desktop.json 설정

```
{
  "mcpServers": {
    "filesystem": {
      "command": "npx",
      "args": [
        "-y",
        "@modelcontextprotocol/server-filesystem",
        "C:/Users/jhkoo/mcp-demo/data"
      ]
    }
  }
}
```

➤ MCP 서버 등록 방법

➤ Filesystem 서버 등록 예시



➤ 실습 프롬프트

- "MCP Filesystem 서버를 통해 tax_sample.csv 파일을 읽고, 상위 5행을 표시하는 예시를 보여주세요."
- "내 data 폴더 안에 있는 모든 CSV 파일 목록을 보여줘."
- "tax_sample.csv 파일에는 몇 개의 행(row)이 있니?"
- "tax_sample.csv 파일에서 평균 소득(income)을 계산해줘."
- "평균 소득과 평균 세금이 포함된 tax_summary.csv 파일을 새로 만들어줘."

➤ Claude Desktop + MCP Filesystem

- 기업/기관 환경에서도 민감 데이터 처리가 가능한 안전한 구조를 제공함

질문	답변
Claude를 통해 프롬프트를 보낼 때 CSV가 회사 쪽으로 전송되나요?	아닙니다. 로컬 MCP 서버를 통해서만 접근합니다.
CSV 내용이 클라우드로 업로드되나요?	업로드되지 않습니다.
Claude Desktop에서만 이런 보호가 되나요?	네, 웹 버전과 달리 로컬에서 MCP 서버를 통해 파일을 처리합니다.

Part 2.

Colab을 활용한 Python 기초 및 Claude 활용 (4H)

Python 개요

Python 개요

➤ Python 개요

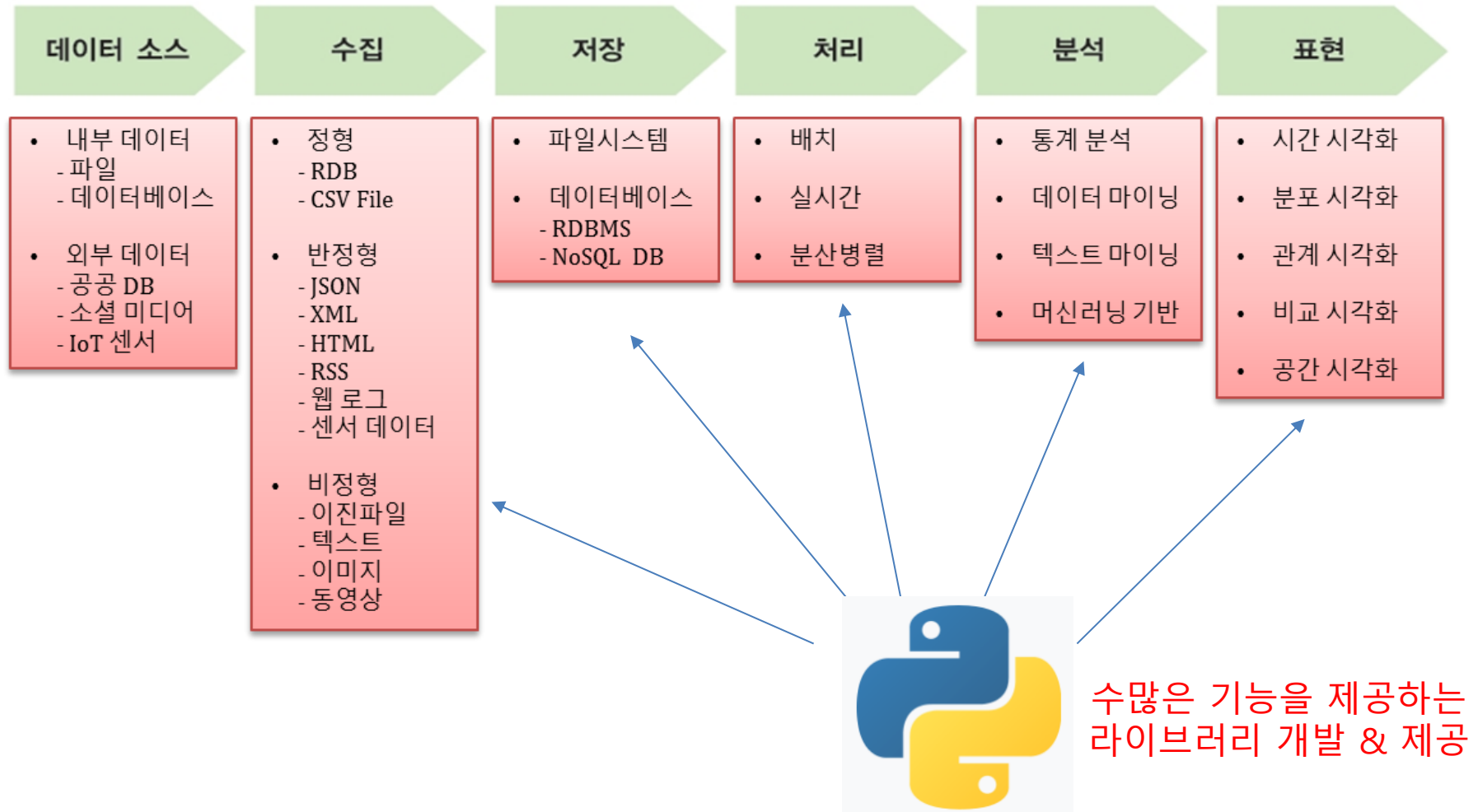
[https://en.wikipedia.org/wiki/Python_\(programming_language\)](https://en.wikipedia.org/wiki/Python_(programming_language))

- ✓ 설계자: Guido van Rossum
- ✓ 개발자: Python Software Foundation
- ✓ 초기개발: 1991년
- ✓ 프로그램 언어
- ✓ Interpreted high-level general-purpose programming language
- ✓ 공식 웹사이트: www.python.org



왜 Python을 사용하는가?

➤ 빅데이터 전주기 프로세스



Python 프로그래밍을 위한 통합개발환경 (IDE)

Integrated Development Environment (IDE)

➤ 대표적인 통합개발환경



- PyCharm: <https://www.jetbrains.com/pycharm/>
- Jupyter: <https://jupyter.org/>
- Visual Studio Code: <https://code.visualstudio.com/>
- Colab: <https://colab.research.google.com/>

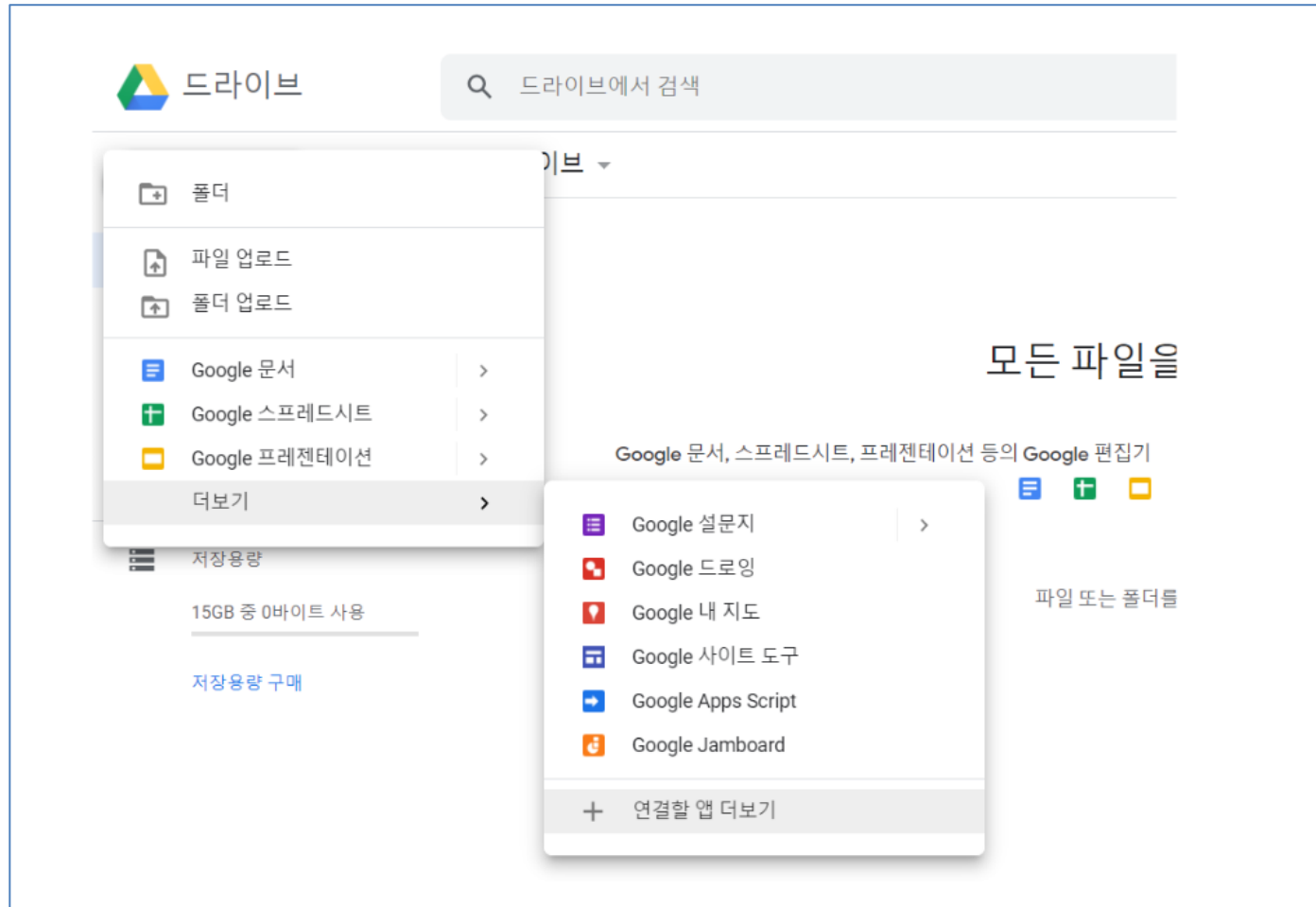
➤ Colab 개요

https://en.wikipedia.org/wiki/Project_Jupyter#Industry_adoption

- ✓ Free Jupyter notebook environment that runs in the cloud and stores its notebooks on Google Drive.
- ✓ 개발자: Google
- ✓ 초기개발: 2018년
- ✓ 공식 웹사이트: <https://colab.research.google.com/>
- ✓ 특징
 - 별도의 Python을 PC에 설치할 필요없이 웹 브라우저를 통해 코드 작성 및 실행 가능

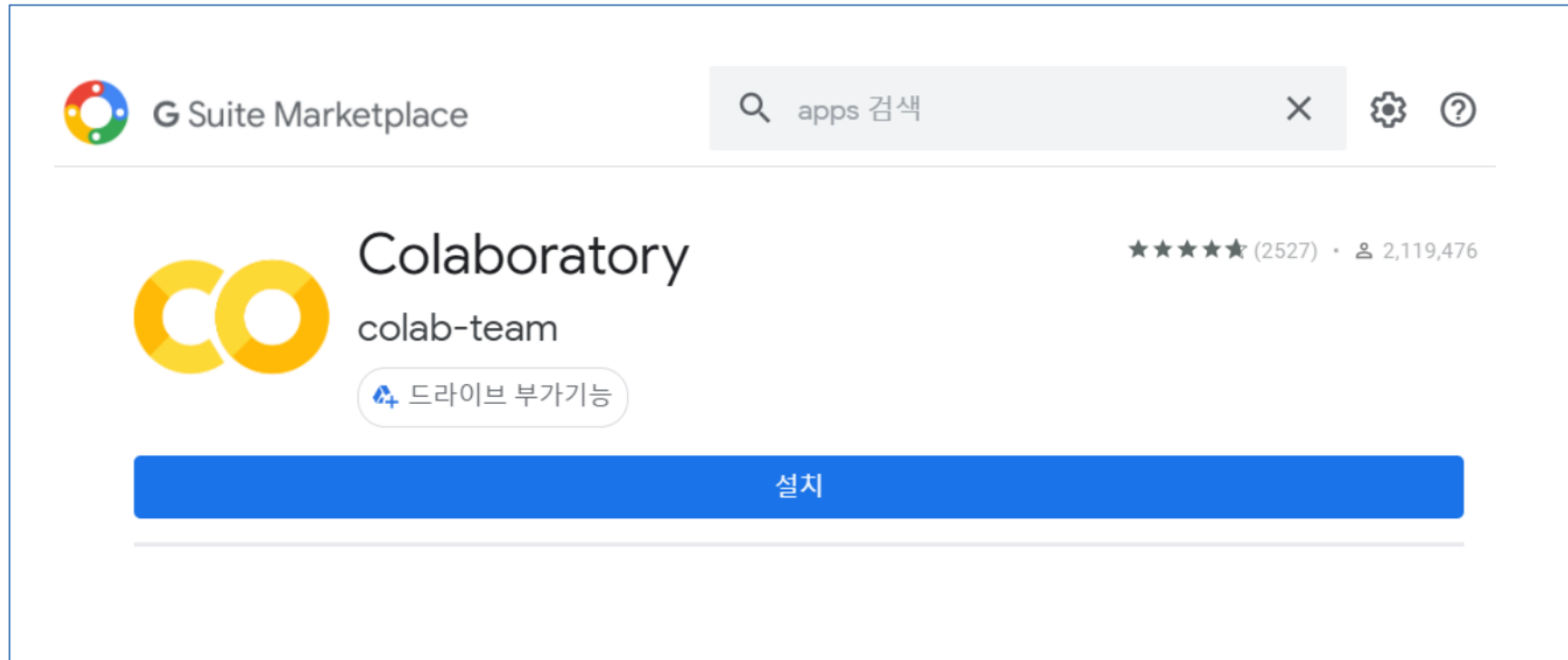
Google Colab 설치 방법 (1/3)

➤ Google Drive에서 새로만들기 > 더보기 > 연결할 앱 더보기 선택



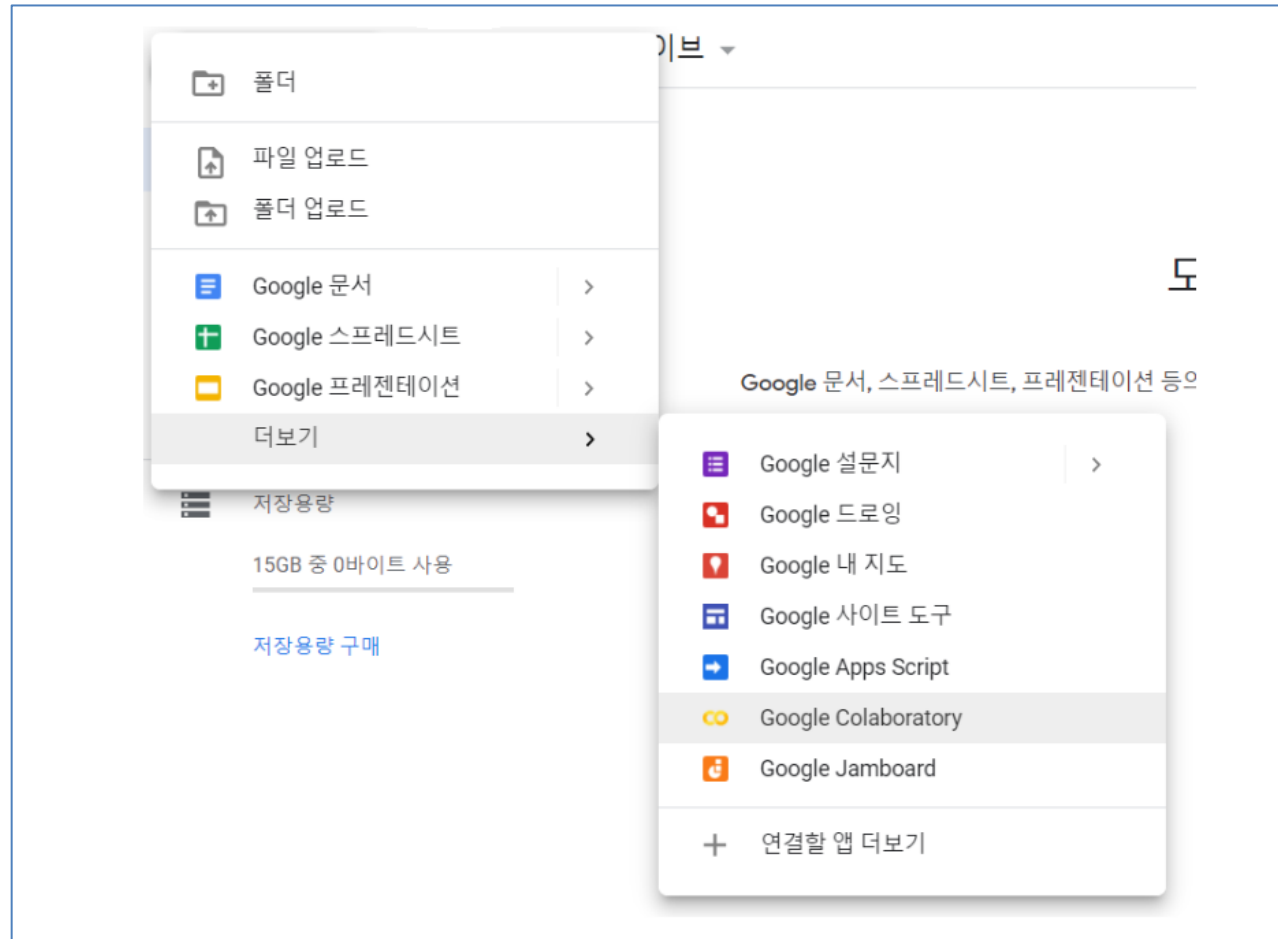
Google Colab 설치 방법 (2/3)

➤ G Suite Marketplace에서 colab 검색 후 설치



Google Colab 설치 방법 (3/3)

➤ 새로 만들기에서 (새 폴더를 만들고) 더보기 > Google Colaboratory 선택

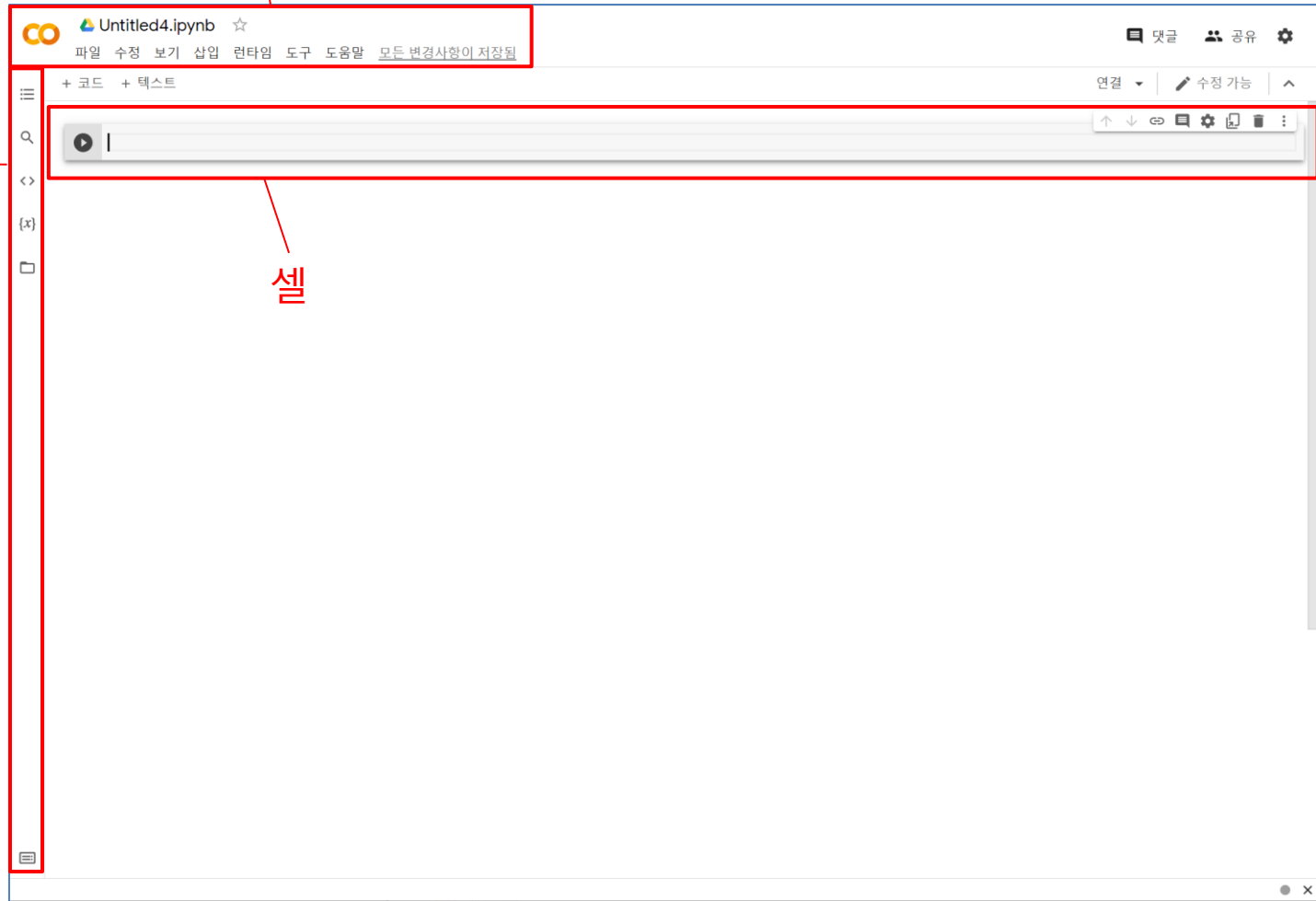


Colab 초기화면 구성

파이썬 노트북 제목 및 메뉴

목차
찾기 및 바꾸기
코드 스니펫
변수
파일

명령 팔레트



➤ 파이썬의 특징

- 쉬운 문법, 강력한 라이브러리
- 데이터 분석, 웹개발, AI 활용 다양
- 행정 데이터 처리에도 적합

➤ 변수와 자료형

➤ 숫자형: int, float

➤ 문자열: str

➤ 자료형 변환 예시

➤ 조건문(if) 기본

➤ if, elif, else 구조

➤ 예시: 소득 구간 분류

➤ 실습 프롬프트

- "소득(income)과 세금(tax) 변수를 선언하고, if-else 문을 이용해 소득 수준을 분류한 뒤 결과를 출력하는 파이썬 예시를 보여줘."

➤ 실습 코드

```
# 소득과 세금 변수 선언
income = 5000
tax = 500

# 소득 수준 분류
if income >= 6000:
    level = "고소득자"
elif income >= 4000:
    level = "중간소득자"
else:
    level = "저소득자"

# 결과 출력
print(f"소득: {income}, 세금: {tax}, 소득 수준: {level}")
```

➤ 반복문(for/while)

- for: 리스트/범위 순회
- while: 조건 만족 시 반복
- 예시: 세금 계산 반복 처리

➤ 리스트(List) 활용

- 여러 값을 저장하는 자료구조
- 인덱싱, 슬라이싱
- 예시: 납세자 이름 목록

➤ 딕셔너리(Dictionary)

➤ key-value 구조

➤ 예시: {"이름":"김철수", "세금":1200}

➤ 검색과 업데이트 용이

➤ 실습 포럼포트

- "for문을 활용해 여러 납세자 데이터(리스트) 반복 처리하고 싶음"

➤ 실습 코드

```
# 납세자 데이터 리스트 (이름, 소득, 세금)
taxpayers = [
    {"name": "김철수", "income": 6000, "tax": 600},
    {"name": "이영희", "income": 4500, "tax": 450},
    {"name": "박민수", "income": 3200, "tax": 320},
    {"name": "정하나", "income": 2500, "tax": 250}
]

# 각 납세자별로 소득 수준 분류
for person in taxpayers:
    income = person["income"]

    if income >= 6000:
        level = "고소득자"
    elif income >= 4000:
        level = "중간소득자"
    else:
        level = "저소득자"

    print(f"{person['name']}님의 소득 수준: {level}")
```

➤ 함수(Function)

- def 키워드로 정의
- 입력값, 반환값 활용
- 예시: 서울 계산 함수

➤ 실습 프롬프트

- "납세자의 이름(name)과 소득(income)을 입력받아 딕셔너리에 저장하고, 그 딕셔너리를 반환하는 파이썬 함수를 작성해줘."

➤ 실습 코드

```
# 납세자 정보를 딕셔너리로 반환하는 함수 정의
def taxpayer_info(name, income):
    taxpayer = {"name": name, "income": income}
    return taxpayer

# 함수 호출 예시
person1 = taxpayer_info("김철수", 5000)
person2 = taxpayer_info("이영희", 4200)

# 결과 출력
print(person1)
print(person2)
```

➤ 실습 프롬프트

- "소득의 10%를 세금으로 계산한 뒤 결과를 CSV 파일로 저장하는 calc_tax.py 스크립트를 MCP Filesystem 서버에 저장해줘."

Claude는 MCP Filesystem 서버를 통해 파이썬 스크립트를 로컬 경로(예: C:/Users/jhkoo/mcp-demo/data/calc_tax.py)에 생성함

➤ 실습 코드

```
import csv

# 입력 데이터: 납세자 이름과 소득
taxpayers = [
    {"name": "김철수", "income": 5000},
    {"name": "이영희", "income": 4200},
    {"name": "박민수", "income": 3000}
]

# 세금 계산 및 CSV 파일 저장
with open("tax_result.csv", "w", newline="", encoding="utf-8") as f:
    writer = csv.writer(f)
    writer.writerow(["name", "income", "tax"])

    for p in taxpayers:
        tax = p["income"] * 0.1 # 세금 = 소득의 10%
        writer.writerow([p["name"], p["income"], tax])

print("✅ tax_result.csv 파일이 성공적으로 생성되었습니다.")
```

➤ 실습 프롬프트

- "calc_tax.py 스크립트를 실행하고 생성된 CSV 파일의 상위 5줄을 보여줘."

Part 3.

데이터 전처리 및 Claude MCP 연동 (3H)

➤ 데이터 전처리란?

- 분석 전 데이터 정리 과정
- 품질 높은 데이터 = 정확한 분석
- "Garbage In → Garbage Out" 원칙

➤ 전처리 필요성

- 오류 데이터 제거
- 결측치(NaN) 처리
- 이상치(Outlier) 보정

➤ 행정 데이터 전처리 사례

- 납세자 소득 값 누락 → 채워 넣기
- 음수/비정상 값 → 제거/수정
- 데이터 형식 통일(숫자 vs 문자열)

➤ 실습 프롬프트

- "납세자 데이터가 들어 있는 CSV 파일을 Pandas로 불러오고, 결측치를 확인한 뒤 결측 행을 삭제하거나 채우는 방법을 예시로 보여줘."

➤ 예시 CSV (파일명: tax_data.csv)

```
name, income, tax  
김철수, 5000, 500  
이영희, ,420  
박민수, 3200,  
정하나, 2500, 250
```

➤ 실습 코드

```
import pandas as pd

# CSV 파일 불러오기
df = pd.read_csv("tax_data.csv")

# 원본 데이터 확인
print("원본 데이터:")
print(df)

# 결측치 확인
print("\n결측치 개수:")
print(df.isnull().sum())
```

```
# 방법 ①: 결측치가 있는 행 제거
df_drop = df.dropna()
print("\n결측치 제거 후:")
print(df_drop)

# 방법 ②: 결측치를 기본값으로 채우기
df_fill = df.fillna({"income": 0, "tax": 0})
print("\n결측치 채운 후:")
print(df_fill)
```

➤ 실습 코드

원본 데이터:

	name	income	tax
0	김철수	5000.0	500.0
1	이영희	NaN	420.0
2	박민수	3200.0	NaN
3	정하나	2500.0	250.0

결측치 개수:

```
name      0
income    1
tax        1
dtype: int64
```

결측치 제거 후:

	name	income	tax
0	김철수	5000.0	500.0
3	정하나	2500.0	250.0

결측치 채운 후:

	name	income	tax
0	김철수	5000.0	500.0
1	이영희	0.0	420.0
2	박민수	3200.0	0.0
3	정하나	2500.0	250.0

➤ 전처리 단계 요약

➤ ① 데이터 불러오기

➤ ② 결측치/이상치 처리

➤ ③ 정규화 및 변환

- `isnull()` : 결측 여부 확인
- `dropna()` : 결측치가 포함된 행 삭제
- `fillna()` : 결측치를 지정 값으로 대체

➤ Pandas 개요

- 파이썬 데이터 분석 라이브러리
- 핵심 객체: DataFrame, Series
- 행정 데이터 분석 필수 도구

➤ 실습 프롬프트

- “납세자 데이터를 포함한 CSV 파일을 불러오고, 특정 열을 선택하며, 조건에 따라 행을 필터링하고, 평균·합계·최댓값 같은 요약 통계를 계산하는 예시를 보여줘.”

➤ 예시 CSV (파일명: tax_data.csv)

```
name, income, tax  
김철수, 5000, 500  
이영희, 4200, 420  
박민수, 3000, 300  
정하나, 2500, 250  
최우성, 6100, 610
```

➤ 실습 코드

```
import pandas as pd

# CSV 파일 불러오기
df = pd.read_csv("tax_data.csv")

# 1 특정 열 선택
print("소득(income) 컬럼만 선택:")
print(df["income"])
print()
```

```
# 2 조건에 따라 행 필터링 (소득이 4000 이상인 납세자)
print("소득이 4000 이상인 납세자:")
print(df[df["income"] >= 4000])
print()

# 3 요약 통계 계산
mean_income = df["income"].mean()
sum_tax = df["tax"].sum()
max_income = df["income"].max()

print(f"평균 소득: {mean_income}")
print(f"총 세금: {sum_tax}")
print(f"최대 소득: {max_income}")
```

➤ 실습 코드

소득(`income`) 컬럼만 선택:

0 5000

1 4200

2 3000

3 2500

4 6100

Name: `income`, dtype: `int64`

소득이 4000 이상인 납세자:

name income tax

0 김철수 5000 500

1 이영희 4200 420

4 최우성 6100 610

평균 소득: 4160.0

총 세금: 2080

최대 소득: 6100

➤ CSV 불러오기

- `pd.read_csv("file.csv")`
- 행/열 구조 확인: `df.head( )`
- 데이터 크기: `df.shape`

➤ 컬럼 선택 & 필터링

➤ 특정 컬럼 추출: `df["컬럼명"]`

➤ 조건 검색: `df[df["컬럼명"] > 값]`

특정 컬럼 선택: `df["income"]`

조건 검색: `df[df["income"] > 4000]`

➤ 기본 연산

➤ 평균: `df["income"].mean( )`

➤ 합계: `df["income"].sum( )`

➤ 최대: `df["income"].max( )`

➤ 최소: `df["income"].min( )`

➤ 실습 프롬프트

- "세금(tax)을 소득(income)으로 나누어 세율(tax_rate) 컬럼을 새로 만들고, 소득 수준별로 납세자를 그룹화한 뒤, 그룹별 평균 세금을 막대그래프로 표시하는 예시를 보여줘."

➤ 실습 코드

```
import pandas as pd
import matplotlib.pyplot as plt

# CSV 파일 불러오기
df = pd.read_csv("tax_data.csv")

# 1 세율(tax_rate) 컬럼 추가
df["tax_rate"] = df["tax"] / df["income"]

# 2 소득 수준별 구간 나누기
def classify_income(income):
    if income >= 7000:
        return "고소득층"
    elif income >= 4000:
        return "중간소득층"
    else:
        return "저소득층"
```

```
df["income_level"] = df["income"].apply(classify_income)

# 3 그룹별 평균 세금 계산
grouped = df.groupby("income_level")["tax"].mean()
print(grouped)

# 4 막대그래프로 시각화
grouped.plot(kind="bar", color="skyblue")
plt.title("소득 수준별 평균 세금")
plt.ylabel("평균 세금")
plt.xlabel("소득 수준")
plt.show()
```

➤ 새로운 컬럼 생성

➤ 계산 기반 파생 변수

➤ 예: `df["tax_rate"] = df["tax"] / df["income"]`

➤ 업무 데이터에서의 활용: 비율, 등급

➤ 그룹화 & 집계

➤ groupby() 활용

➤ 예: 지역별 평균 소득

➤ `df.groupby("region")["income"].mean( )`

➤ 집계 함수: mean, sum, count

➤ 시각화 기초

➤ Matplotlib 기본 그래프

➤ 막대/선 그래프 생성

➤ `df.groupby("region")["tax"].mean( ).plot(kind="bar")`

➤ 시각화를 통한 인사이트 도출

➤ MCP API 서버 활용

- 공공데이터 API 호출 → JSON → DataFrame 변환
- 실습 예: 국세통계 데이터 가져오기

➤ 실습 프롬프트

- "MCP API 서버를 사용하여 세무 통계 샘플 데이터를 JSON 형식으로 가져오고, 이를 Pandas DataFrame으로 변환한 뒤, 전처리(cleaning) 과정을 거쳐 CSV 파일로 MCP Filesystem에 저장하는 예시를 보여줘."

➤ 실습 코드

```
import pandas as pd
import requests

# 1 MCP API 서버에서 세무 통계 JSON 데이터 가져오기
url = "https://example-mcp-server/api/tax-stats" # MCP API 예시 URL
response = requests.get(url)
data = response.json()

# JSON 예시 구조
# [
#   {"name": "김철수", "income": 5000, "tax": 500},
#   {"name": "이영희", "income": 4200, "tax": 420},
#   {"name": "박민수", "income": None, "tax": 300}
# ]
```

➤ 실습 코드

```
# 2 DataFrame으로 변환
```

```
df = pd.DataFrame(data)
```

```
# 3 결측치 처리
```

```
df["income"] = df["income"].fillna(df["income"].mean())
```

```
# 4 세율 컬럼 추가
```

```
df["tax_rate"] = df["tax"] / df["income"]
```

```
# 5 CSV 파일로 MCP Filesystem에 저장
```

```
df.to_csv("tax_stats_cleaned.csv", index=False, encoding="utf-8")
```

```
print("✅ tax_stats_cleaned.csv 파일이 성공적으로 생성되었습니다.")
```

➤ 행정 업무 적용 예시

➤ 국세 데이터 자동 정리 파이프라인

➤ “데이터 수집 → 전처리 → 저장” 일괄 처리

Part 4.

Excel VBA 자동화 실습 (4H)

➤ 정의

- VBA는 Microsoft Office(특히 Excel)에서 반복적인 작업을 자동화하기 위한 프로그래밍 언어

➤ 특징

- Excel 내부에서 매크로로 실행
- Python처럼 구조적 프로그래밍 가능 (If, For, Function 등)
- 데이터 보고서 생성, 셀 처리, 파일 저장 등을 자동화

➤ 코드 예시

```
Sub HelloVBA()  
    MsgBox "안녕하세요! VBA 자동화 실습입니다."  
End Sub
```

➤ 주석

- Sub ~ End Sub : 하나의 프로그램 단위
- MsgBox : 메시지 박스를 띄우는 명령어

➤ 매크로 실행 기본

- Excel 상단 메뉴 → [개발 도구] → [Visual Basic] 클릭
- 모듈 삽입: Insert → Module
- 코드 입력 후 F5 키로 실행
- Excel 창에서 매크로 실행 확인

➤ 보안 설정

- 경고가 나올 경우:
 - 파일 > 옵션 > 보안 센터 > 매크로 설정 > 모든 매크로 사용 선택

➤ 셀 값 읽기와 쓰기

```
Sub CellExample()  
    Dim income As Double  
    income = Cells(2, 3).Value           'C2 셀 읽기  
    Cells(2, 4).Value = income * 0.1     '세금 계산 결과 D2에 저장  
End Sub
```

➤ 주식:

- Cells(행, 열) 형식으로 셀 접근
- 실무에서 "세월 계산, 합계 보고서" 등 자동 계산에 사용

➤ 반복문을 이용한 자동 계산

```
Sub LoopTaxCalc()  
    Dim i As Integer  
    For i = 2 To 10  
        Cells(i, 4).Value = Cells(i, 3).Value * 0.1  
    Next i  
End Sub
```

➤ 주석:

- For i = 2 To 10 : 2행부터 10행까지 반복
- 세금 자동 계산, 보고서 자동 완성에 활용

➤ 조건문을 이용한 분류

```
Sub TaxCategory()  
    Dim i As Integer  
    For i = 2 To 10  
        If Cells(i, 3).Value >= 5000 Then  
            Cells(i, 5).Value = "고소득"  
        Else  
            Cells(i, 5).Value = "일반"  
        End If  
    Next i  
End Sub
```

➤ 주석:

- If ~ Else 문으로 조건별 처리세무 데이터의 소득 구간별 분류 등 행정 데이터 분류 자동화

➤ 자동 보고서 생성

```
Sub CreateReport()  
    Dim ws As Worksheet  
    Set ws = Sheets.Add  
    ws.Name = "세무요약보고서"  
    ws.Range("A1").Value = "소득세 요약 보고서"  
    ws.Range("A2").Value = "작성일: " & Date  
    ws.Range("A4").Value = "총 납세자 수"  
    ws.Range("B4").Value = WorksheetFunction.CountA(Sheets("Data").Range("A2:A100"))  
End Sub
```

➤ 주석:

- 새 시트를 생성하고, 요약 정보를 자동 입력
- 실무 보고서 자동 생성 템플릿의 기초

➤ 개념

- Claude MCP를 이용하면 VBA 코드 초안을 Claude가 자동으로 작성 가능
- Claude에 다음과 같은 프롬프트 입력:
 - **“세울 계산 및 보고서 자동화를 수행하는 Excel VBA 코드를 작성해줘.”**

```
Sub AutoTaxReport()  
    Dim ws As Worksheet  
    Set ws = ThisWorkbook.Sheets("TaxData")  
    ws.Range("E1").Value = "세금계산"  
    For i = 2 To 100  
        ws.Cells(i, 5).Value = ws.Cells(i, 3).Value * 0.1  
    Next i  
End Sub
```

➤ Step

1. Colab에서 세무 데이터를 전처리 (tax_data.csv 생성)
2. Claude MCP가 이 파일을 Excel로 전달
3. VBA가 자동으로 요약 보고서 생성
4. Streamlit으로 보고서 결과 시각화

Part 5.

Colab + Streamlit을 활용한 웹앱 구현 (3H)

➤ Streamlit 개요

- 파이썬 기반 웹앱 프레임워크
- 간단한 코드 → 웹 대시보드 변환
- 데이터 분석가, 비개발자도 쉽게 활용

➤ 실습 프롬프트

- "제목(title), 설명(description), 그리고 세무(tax) 데이터 샘플 테이블을 표시하는 간단한 Streamlit 앱 예시를 보여줘."

➤ 실습 코드 (app_basic.py)

```
import streamlit as st
import pandas as pd

# 1 앱 제목과 설명
st.title("세무 데이터 대시보드")
st.write("이 앱은 납세자 소득과 세금 데이터를 간단히 표시합니다.")

# 2 샘플 데이터 생성
data = {
    "이름": ["김철수", "이영희", "박민수", "정하나"],
    "소득": [5000, 4200, 3200, 2500],
    "세금": [500, 420, 320, 250]
}
```

➤ 실습 코드 (app_basic.py)

```
df = pd.DataFrame(data)

# 3 테이블 표시
st.subheader("샘플 납세자 데이터")
st.table(df)
```

- 터미널 또는 Claude MCP Filesystem 폴더 경로에서 다음 명령 실행:
 - streamlit run app_basic.py

➤ Streamlit 설치 & 실행

- 설치: `pip install streamlit`
- 실행: `streamlit run app.py`
- 브라우저에서 결과 확인

➤ 기본 구성요소

➤ `st.title( )`, `st.write( )`, `st.dataframe( )`

➤ 텍스트, 표, 차트 출력

➤ 예시: 간단한 국세 데이터 표

➤ 실습 프롬프트

- "Streamlit 앱을 확장하여 CSV 파일을 불러오고, 데이터를 테이블로 표시하며, 소득(income)을 막대그래프로 시각화하는 예시를 보여줘."

➤ 실습 코드 (app_csv_chart.py)

```
import streamlit as st
import pandas as pd

# 1 제목과 설명
st.title("세무 데이터 시각화 앱")
st.write("CSV 파일을 업로드하여 납세자 데이터를 테이블과 막대그래프로 표시합니다.")

# 2 CSV 파일 업로드 위젯
uploaded_file = st.file_uploader("CSV 파일을 업로드하세요", type=["csv"])
```

➤ 실습 코드 (app_csv_chart.py)

```
if uploaded_file is not None:
    # 3 CSV 파일 읽기
    df = pd.read_csv(uploaded_file)

    # 4 데이터 테이블 표시
    st.subheader("납세자 데이터 미리보기")
    st.dataframe(df)

    # 5 소득(income) 막대그래프 표시
    if "income" in df.columns:
        st.subheader("소득 막대그래프")
        st.bar_chart(df["income"])
    else:
        st.warning("⚠ 'income' 컬럼이 CSV 파일에 없습니다.")
else:
    st.info("📄 CSV 파일을 업로드하면 데이터가 여기에 표시됩니다.")
```

➤ 실행 방법

➤ 터미널에서 실행:

➤ `streamlit run app_csv_chart.py`

➤ 챗봇 구조 이해

- 입력 → AI 응답 → 출력
- Claude API 개요
- 대화형 질의 응답 시스템

➤ 실습 프롬프트

- "사용자의 질문을 Claude API로 전송하고, 그 응답을 출력하는 간단한 파이썬 함수를 만드는 방법을 보여줘."

➤ 실습 코드 (ask_claude.py)

```
import requests
import json

# 1 Claude API 키 설정 (환경변수 또는 직접 입력 가능)
API_KEY = "YOUR_CLAUDE_API_KEY"
API_URL = "https://api.anthropic.com/v1/messages"
```

➤ 실습 코드 (ask_claude.py)

2 Claude 에 질문을 보내는 함수 정의

```
def ask_claude(question):
```

```
    headers = {
```

```
        "Content-Type": "application/json",
```

```
        "x-api-key": API_KEY,
```

```
        "anthropic-version": "2023-06-01"
```

```
    }
```

```
    data = {
```

```
        "model": "claude-3-sonnet-20240229",
```

```
        "max_tokens": 200,
```

```
        "messages": [
```

```
            {"role": "user", "content": question}
```

```
        ]
```

```
    }
```

Claude API 키를 헤더에 포함

messages 필드에 사용자의 질문 전달

➤ 실습 코드 (ask_claude.py)

```
response = requests.post(API_URL, headers=headers, data=json.dumps(data))
```

requests.post() 로 Claude API 호출

```
if response.status_code == 200:
```

```
    answer = response.json()["content"][0]["text"]
```

```
    print("Claude의 응답:")
```

```
    print(answer)
```

응답(response.json())에서 텍스트 추출 후 출력

```
else:
```

```
    print("요청 실패:", response.status_code, response.text)
```

3 함수 실행 예시

```
if __name__ == "__main__":
```

```
    user_question = "파이썬에서 리스트와 딕셔너리의 차이점을 설명해줘."
```

```
    ask_claude(user_question)
```

➤ Claude API 연동 절차

- API 키 등록
- 요청(Request) & 응답(Response) 구조
- 간단한 텍스트 응답 출력

➤ 실습 프롬프트

- "사용자가 세무 관련 질문을 입력하면 Claude API로 전송하고, 응답을 화면에 표시하는 Streamlit 기반 챗봇을 만들어줘."

➤ 실습 코드 (tax_chatbot.py)

```
import streamlit as st
import requests
import json
import os

# 1 Claude API 키 설정 (환경 변수 사용 권장)
API_KEY = os.getenv("ANTHROPIC_API_KEY") or "YOUR_CLAUDE_API_KEY"
API_URL = "https://api.anthropic.com/v1/messages"
```

➤ 실습 코드 (tax_chatbot.py)

```
# 2 Claude API 호출 함수 정의
def ask_claude(question):
    headers = {
        "Content-Type": "application/json",
        "x-api-key": API_KEY,
        "anthropic-version": "2023-06-01"
    }

    data = {
        "model": "claude-3-sonnet-20240229",
        "max_tokens": 300,
        "messages": [{"role": "user", "content": question}]
    }
```

➤ 실습 코드 (tax_chatbot.py)

```
response = requests.post(API_URL, headers=headers, data=json.dumps(data))
if response.status_code == 200:
    return response.json()["content"][0]["text"]
else:
    return f"⚠ 요청 실패: {response.status_code}"

# 3 Streamlit 앱 UI
st.title("💬 Claude 세무 챗봇")
st.write("세금 계산, 신고 기준 등 궁금한 점을 Claude에게 물어보세요.")

# 4 사용자 입력창
user_question = st.text_input("세무 관련 질문을 입력하세요:")
```

➤ 실습 코드 (tax_chatbot.py)

```
# 5 버튼 클릭 시 Claude 응답 출력
if st.button("질문 보내기"):
    if user_question.strip():
        with st.spinner("Claude가 답변을 생성 중입니다..."):
            answer = ask_claude(user_question)
            st.subheader("📄 Claude의 답변:")
            st.write(answer)
    else:
        st.warning("질문을 입력해주세요.")
```

➤ streamlit run tax_chatbot.py

➤ 실습 예시

- “세금 신고 기한은 언제인가요?” 입력
- Claude 응답 출력하기

➤ Streamlit 입력창 구성

➤ `st.text_input("질문 입력")`

➤ 사용자의 행정 질문 받기

➤ Claude API 응답 출력

- 입력값 전달 → Claude 응답 표시
- Streamlit 내 실시간 대화

➤ 챗봇 적용 사례

- 세무 Q&A 챗봇 예시
- "소득세율은 얼마인가요?"
- "부가가치세 신고 절차는?"

Part 6.

Streamlit 기반 챗봇 구현 (4H)

Part 7.

데이터 대시보드 구축 (3H)

➤ 데이터베이스(DB) 개요

- 행정 데이터는 대량, 구조화된 형태
- DB 활용 → 빠른 검색, 안정적 관리
- SQLite: 가볍고 실습에 적합

➤ Python-SQLite 연동

- `sqlite3.connect("tax.db")`
- SQL 질의 실행 예시: `SELECT * FROM taxpayer`
- DataFrame으로 변환

➤ MCP DB 서버 활용

- Claude Desktop ↔ MCP DB 연동
- SQL 질의 → 응답 확인
- 행정 데이터 조회 자동화

➤ 실습 예시

- 납세자 데이터 DB 구축
- Streamlit에서 검색 및 출력

➤ 대시보드 개념

- 다양한 지표를 한 화면에 통합
- 실시간 분석 & 시각화 제공
- 국세 데이터 대시보드 예시

➤ Streamlit 확장 기능

- `st.selectbox()` 필터 기능
- `st.sidebar()` 옵션 메뉴
- 사용자 맞춤형 인터페이스

➤ 실습 과제

- 연도별/지역별 소득세 통계 표시
- 필터링 + 차트 결합

➤ 행정업무 자동화 필요성

- 반복 보고서 → 시간 소모
- 데이터 기반 자동화 → 효율성 ↑
- Claude MCP로 자동화 가능

➤ 자동화 사례

- 정기 보고서 자동 생성
- 세금 납부 현황 요약
- 이메일 발송 자동화

➤ 적용 방안

- MCP Filesystem: 보고서 저장
- MCP API: 메일 발송 연동
- Streamlit UI로 실행 버튼 제공

Part 8.

미니프로젝트

(3H)

➤ 프로젝트 개요

➤ 팀별 주제 선정

➤ 목표: 데이터 → 전처리 → 웹앱/챗봇/대시보드 통합

➤ 주제 예시

- 국세 신고서 자동 작성 앱
- 세법 검색 챗봇
- 자동 보고서 생성 대시보드

➤ 진행 방식

- 팀별 설계 및 구현 (60분)
- Claude MCP 활용 의무화
- 발표 준비

➤ 발표 & 피드백

- 팀별 시연 발표
- 강사 및 동료 피드백
- 적용 가능성 토론

➤ QR 코드

